Lambda functions

→ require lambda keyword
→ can only have one expression

eg: `lambda a,b : a+b`

arguments returns operation.

usage → `addit = lambda a, b: a+b`

`print(addit(5,7))`

⇒ 12

`check_even = lambda x : x % 2 == 0`

→ cannot use if.
→ == is already a comparison check on its own.

→ returns True if x is even

Map functions → used to apply transformations.

→ applies a function to all items in a list, (or any other iterable)

eg num = [1, 2, 3, 4]

```
num1 = list (map (lambda x: x**2 , num))
```

converting to list — function — list

⇒ num1 = [1, 4, 9, 16]

→ returns a map by default, & have to be converted into a list etc.

Multiple iterables.

num1 = [1, 2, 3] ; num2 = [4, 5, 6]

```
result = list (map (lambda x, y: x + y , num1, num2))
```

list conversion — map — 2 parameters — 2 lists

⇒ result = [5, 7, 9]

Q → Convert list of strings into number

str1 = ['1', '2', '3']

int1 = list (map (int, str1))

↓

type casting ~~into~~ into int

(*) no parenthesis

⇒ int1 = [1, 2, 3]

Q words = ['a', 'b', 'c']

upp = list (map (str.upper, words))

⇒ upp = ['A', 'B', 'C']

(**) string function without parenthesis

Q → To return from a dict().

```
def get_name (x) :
    return x['name']
```

dict of dictionaries

peps = [{'name': 'Viv', 'age' = 3}, {'name' = 'H', 'age' = 2}]

res = list (map (get_name, peps))

⇒ res = ['Viv', 'H']

Filter functions

→ used to filter items from a iterable based on a condition.

~~num = [1, 2, 3, 4]~~ ~~even = list~~ → also need to convert to list.

```
nums = [1, 2, 3, 4, 5]
```

✗ `even = filter(lambda x: x%2==0, nums))`

filter() — function — list

✓ `even = list(filter(lambda x: x%2==0, nums))`

convert to list — filter based on condition

In above scenario, if we put map() in place of filter

`even_check = list(map(lambda x: x%2==0, nums))`

⇒ check = [False, True, False, True, False]

even = [2, 4]

~~eg of filter()~~

eg of filter()

(check >5 & 10<)

nums = [1, 2, 3, 4, 5, 6]

check_greater than_5 and less than 10 =

= list(filter(lambda x: x > 5 and x < 10, nums))

=> check... = [6]

---

(9) → Filter names of people above 10.

peeps = [{'name': 'Viv', 'age': 11},
{'name': 'Abc', 'age': 12},
{'name': 'xyz', 'age': 9}]

old_peps = list(filter(lambda x: x['age'] > 10, peeps))

print(list(x['name'] for x in old_peps))

=> ['Viv', 'Abc']